

最高のシステムエンジニアリングを目指して

作成 平野直彦
日付 1994. 9. 28

目

次

1. 最高のシステムエンジニアリングを目指して	・ ・ ・	1
2. 戦略を成功に導く為の手段	・ ・ ・	2
3. ソフトウェアエンジニアリング環境	・ ・ ・	3
4. 方法論	・ ・ ・	4
5. ツール	・ ・ ・	6
6. リエンジニアリング	・ ・ ・	8
7. 再利用可能性	・ ・ ・	1 0
8. 人材	・ ・ ・	1 1
9. 管理	・ ・ ・	1 2

最高のシステムエンジニアリングを目指して

システム開発
の目指すもの

短期開発

低コスト

高品質

高生産性

現在の問題

ビジネスニーズに対して開発期間が長い
経営戦略に対してシステムがマッチしていない
手作業で作業している為開発コストが高い
ドキュメントが文章の為理解しづらい
ドキュメントの枚数が多い為書き漏れ発生
システムができてドキュメントがない
プログラムがわかりにくくメンテナンス不可
e t c

解決方法

方法論

ツール

リエンジニアリング

再利用可能性

人

材

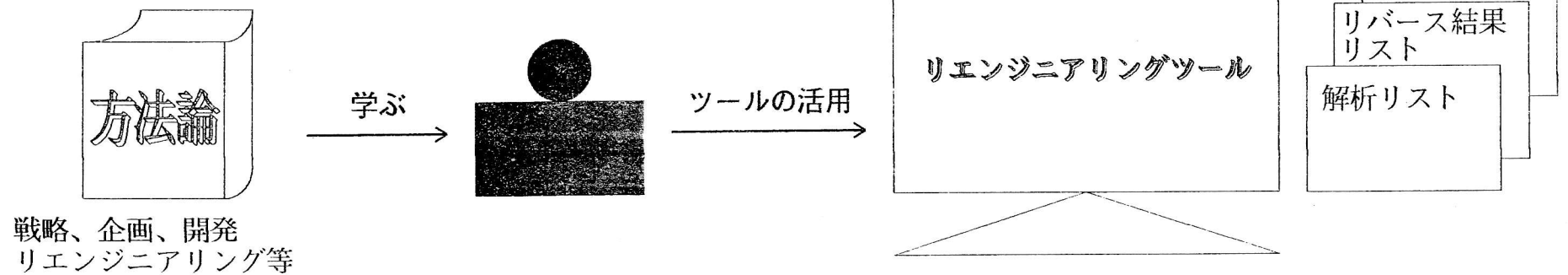
管

理

戦略を成功に導く為の手段

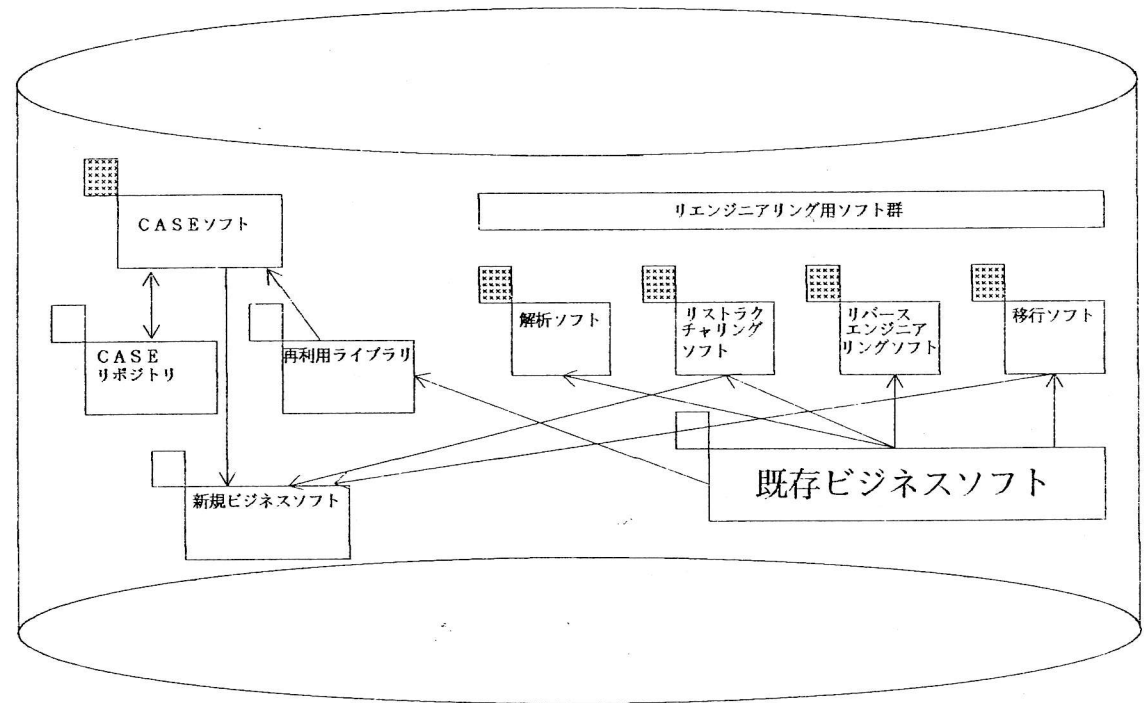
手 段	必 要 性	内 容
方法論	・ビジネスニーズにあったシステムを導く為のガイド ・より効果的にシステムを構築する為のガイド（CASEを使用すうえでの、前提となる）	・情報戦略から構築までを導き為の方法論を検討する。
ツール	・方法論をより正確に実行する為 ・低コストを実現する為には、機械化がいる ・短期開発をする為	I-CASE 方法論にあった統合化ケースツール
リエンジニアリング	・保守のための自動化支援 ・システムの理解を高める	・解析 ・リストラクチャリング ・リバースエンジニアリング ・移行
再利用可能性	・高生産性 ・高品質 ・短期開発 ・低コスト これらの実現の為	・CASEツール内において管理されているリポジトリから再利用 ・プログラム・ライブラリから
人材	・優れた方法論、ツールを効果的に使用する為 例えば、外洋レースでは優れた船を持つことが必要であるが、それだけではレースに勝つことができない。高度な教育を受けた人材から構成されるチームが必要である。	・プロジェクト・マネージャー ・システム・アナリスト ・CASEエンジニア等
管理	・最新の方法論、ツール、人材を用いる為、従来の管理とは異なる仕方を行う。	・手段となる要素 ・動機づけ ・計量方法の明確化等

ソフトウェアエンジニアリング環境



システム・アナリストやシステム・エンジニアは方法論を学びそれを土台として、ツールを活用し、短期開発、低コスト、高品質を目指す。

上記により管理方法も変革される。



方法論

一連のタスクを組み合わせて使用し、作業者を結論に導く
為のガイドである。

必要性

一貫した考えや規準及び明確なダイアグラムがなかった為に
混乱が生じてきた。

Oxford大学 C. A. Hoare教授談

「あいまいな基礎の上に、ソフトウェア工学の作業規律を適用すると、きっと悲惨な結果をまねくであろう。それは、化学の基礎を燃素の上に置いたり、天文学の基礎を地球が平板だという仮説の上に置くのと同様である。」

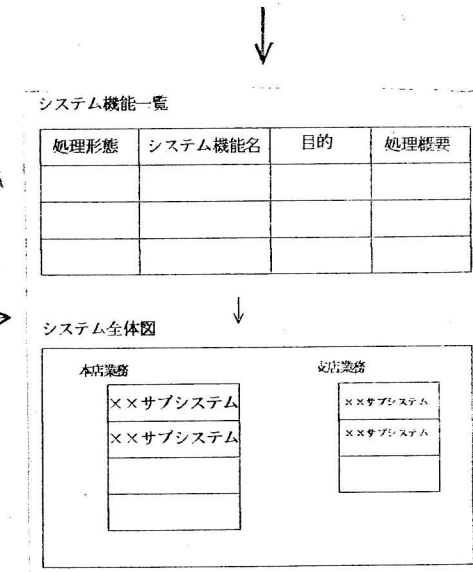
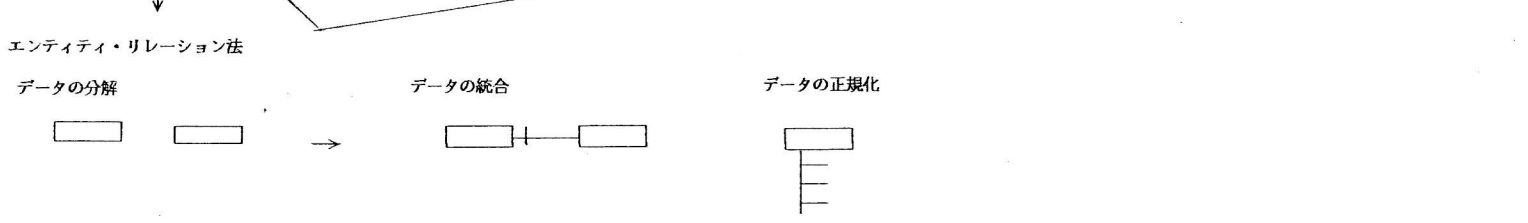
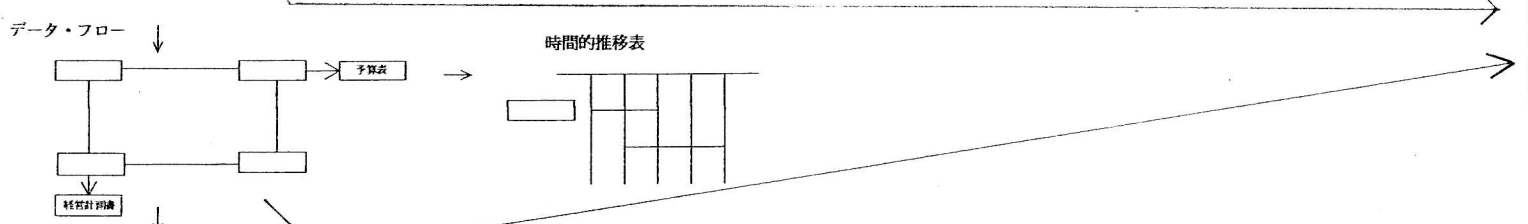
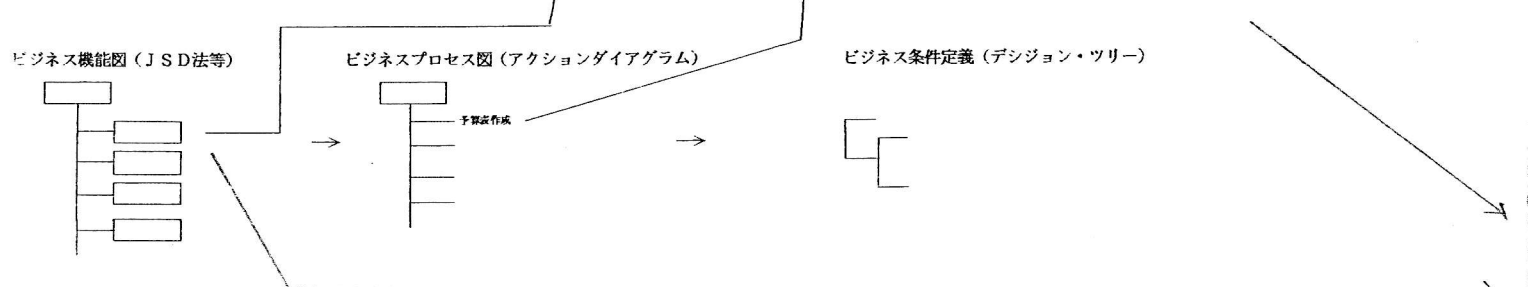
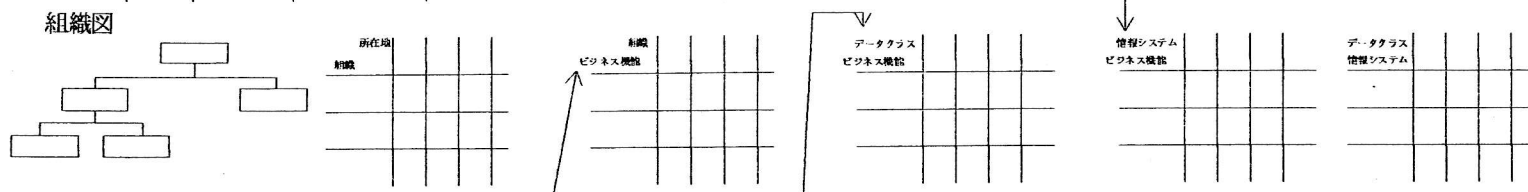
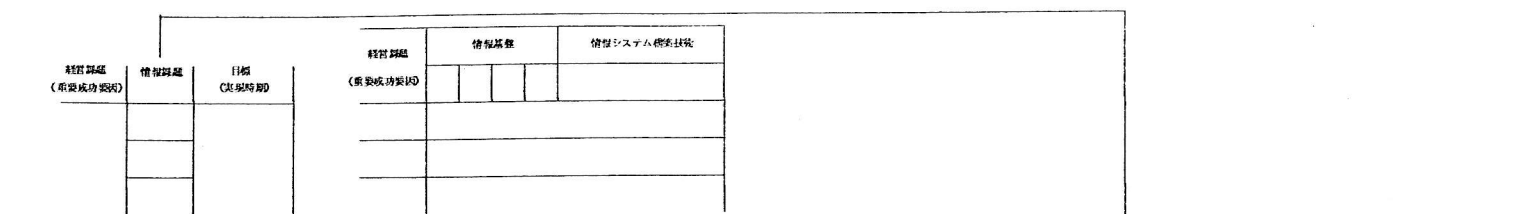
方法論の種類

戦略

- ・ 5つの競争要因、3つの基本戦略、価値連鎖分析
- ・ 戦略的グリッド
- ・ 重要成功要因分析
- ・ S W O T 分析等

設計 (注 IEは戦略から構築までの流れでできている)

- ・ I E (インフォメーション・エンジニアリング)
- ・ S S A D M
- ・ オブジェクト指向設計
- ・ J S D法等



ツール

CASEツールはコンピュータを使用して、システムの企画から構築、及びプログラム作成迄を支援するシステムである。

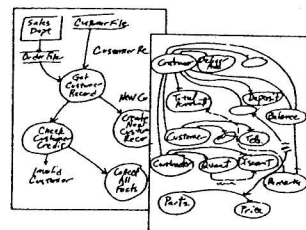
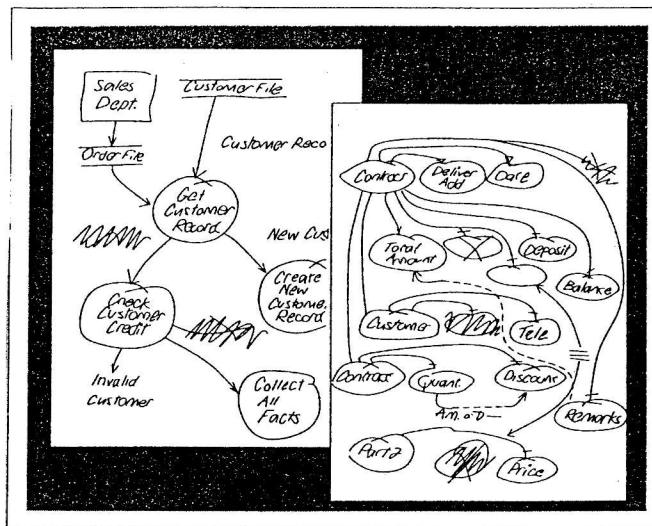
必要性

- ・手書きのダイアグラムでは数多くのエラー、非一貫性、抜けが残っていた。
- ・設計書は膨大なページになり、手作業に関連のチェックを行うのは困難
- ・プログラム作成も非生産性であった。

効果

- ・システムの開発を短期間で行うことができる。
- ・設計情報やプログラム・コードの作成によって、大幅な経費節減を可能にする。
- ・一連のチェックをコンピュータで行う為、品質の確保向上をはかれる。

従来の環境



改革



新しい環境

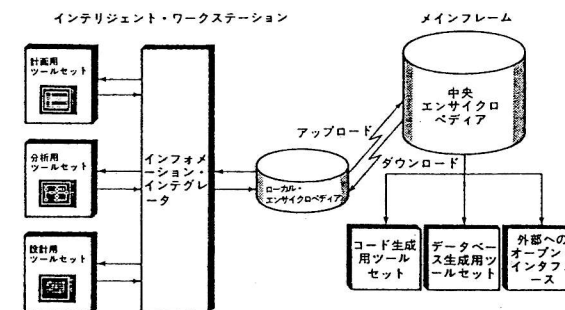


図 2.17 Texas Instruments 社の IE 用ツールセット

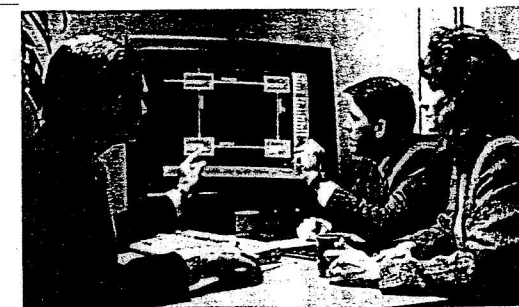
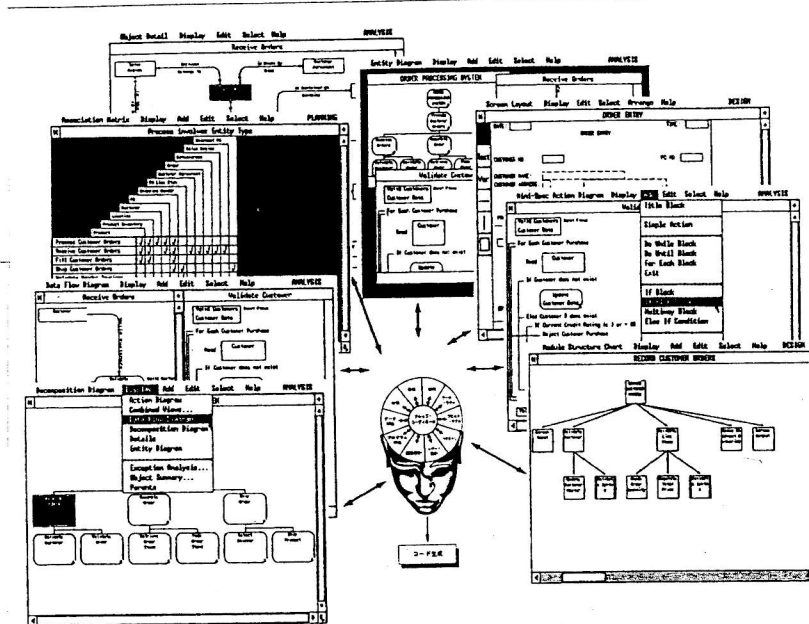
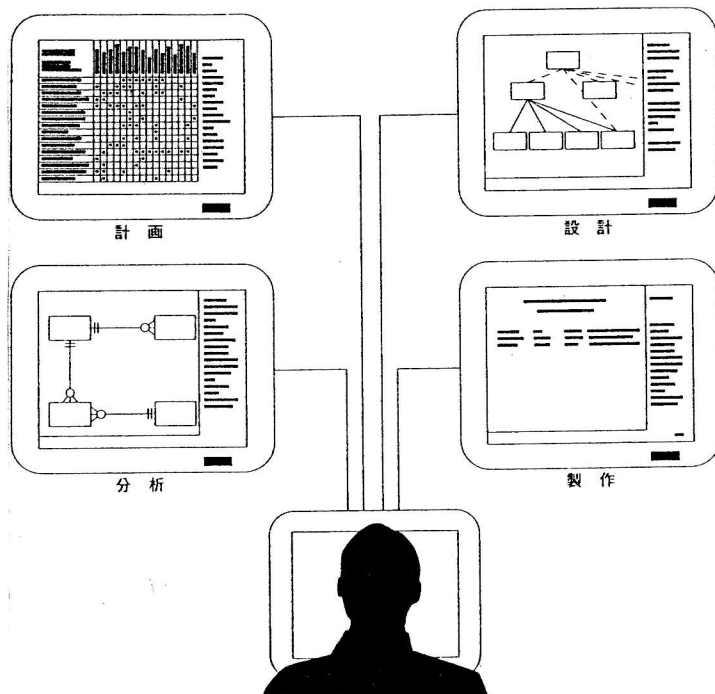


図 2.1 システム・アナリストは、インフォメーション・エンジニアリング用のワークベンチを使って仕事をする。(Texas Instruments 社の IEF)

リエンジニアリング

リエンジニアリングは自動化ツールの支援により、既存ソフトウェアシステムを調査したり、修正したりすることである。

リエンジニアリングの種類と目的

- | | |
|---------------|-----------------------------------|
| ・ 解析 | ・ ・ 既存システムを調べる。システムの品質を測定する |
| ・ リストラクチャリング | ・ ・ ソフトウェアの形式を変更する。 |
| ・ リバーエンジニアリング | ・ ・ システム・ドキュメントを再生し、設計情報を明らかにする |
| ・ 移行 | ・ ・ ある言語、ある環境へそれぞれ変換する。また新しい技術へ変換 |

必要性

- ・ ソフトウェアのバックログの増大
- ・ システム・ドキュメントが貧弱
- ・ システムの理解が困難であり変更が危険
- ・ 壊れやすいソフトウェア

効果

- ・ 保守しやすさの改善
- ・ 高い技術への移行
- ・ システム寿命の延長
- ・ C A S E ツールが支援するリポジトリ内への構成要素の取り込

リエンジニアリングの例

- ・ リストラクチャリングツールを使用してプログラムを構造化した例

Original FORTRAN-IV

```

SUBROUTINE ORDONE(PRI, IC, KAS, RC)
C
C IF PRI IS TRUE, SUBROUTINE ORDONE ORDERS ELEMENTS
C OF VECTOR KAS IN INCREASING ORDER OF THEIR VALUES
C AND VECTOR RC CORRESPONDINGLY. IF PRI IS FALSE,
C SUBROUTINE ORDONE ORDERS ELEMENTS OF VECTOR RC IN
C DECREASING ORDER OF THEIR VALUES AND VECTOR KAS
C CORRESPONDINGLY.

```

```

LOGICAL PRI
DIMENSION RC(IC), KAS(IC)
ID = IC2
IF (ID GE 0) RETURN
MO = ID + IC
DO 40 J=1,MO
  I=J
  IF (LT 1) GO TO 40
  JJ = I - ID
  IF (PRI) GO TO 50
  IF (RC(I) GE RC(JJ)) GO TO 40
  G = RC(I)
  RC(I) = RC(JJ)
  RC(JJ) = G
  I = KAS(I)
  KAS(I) = KAS(JJ)
  KAS(JJ) = I
  I = I - ID
  GO TO 20
40 CONTINUE
ID = ID2
GO TO 10
50 IF (KAS(I) LE KAS(JJ)) GO TO 40
GO TO 30
END

```

Standard FORTRAN-77 Output

```

C Structured by FOR-STRUCT, v1.0.1
C Options SET a-7
SUBROUTINE ORDONE(PRI, IC, KAS, RC)
C
C IF PRI IS TRUE, SUBROUTINE ORDONE ORDERS ELEMENTS
C OF VECTOR KAS IN INCREASING ORDER OF THEIR VALUES
C AND VECTOR RC CORRESPONDINGLY. IF PRI IS FALSE,
C SUBROUTINE ORDONE ORDERS ELEMENTS OF VECTOR RC IN
C DECREASING ORDER OF THEIR VALUES AND VECTOR KAS
C CORRESPONDINGLY.
LOGICAL PRI
DIMENSION RC(IC), KAS(IC)
ID = IC2
IF (ID GE 0) RETURN
MO = ID + IC
DO 40 J=1,MO
  I=J
  IF (LT 1) THEN
    IF (PRI) THEN
      IF (KAS(I) LE KAS(JJ)) GOTO 51
    ELSE IF (RC(I) GE RC(JJ)) THEN
      GOTO 51
    ENDIF
    G = RC(I)
    RC(I) = RC(JJ)
    RC(JJ) = G
    I = KAS(I)
    KAS(I) = KAS(JJ)
    KAS(JJ) = I
    I = I - ID
    GOTO 20
  51 CONTINUE
  40 CONTINUE
ID = ID2
GOTO 10
END

```

ADD-PROJ.

```

IF PROJ-CODE AND NEW-PROG NEXT
SENTENCE ELSE
GO TO ADD-PROG.
IF INDX-P IS EQUAL TO 100 GO TO
WRITE-REC.
MOVE EXP-REV-PROJ TO PROJ-TABL (INDX-P).
MOVE NO-PROG-OR-TE TO MODE-T (INDX-P).
ADD 1 TO INDX-P.

```

図 5.7 構造化されていない COBOL コードの例。複数のエントリーポイントとイグジットポイントがあり、パラグラフ外への GO TO 分岐もある。

プログラムロジックのリストラクチャリング・ツール

0960-RETOOL.

*PERFORMED BY 0940-WRITE-GOOD-RECORD.

```

IF PROJ-CODE AND NEW-PROJ
IF INDX-P IS EQUAL TO 100
PERFORM 0980-WRITE-REC
ELSE
MOVE EXP-REV-PROJ TO PROJ-TABL (INDX-P)
MOVE NO-PROG-OR-TE TO MODE-T (INDX-P)
ADD 1 TO INDX-P
PERFORM 0970-ADD-PROG
PERFORM 0980-WRITE-REC
ELSE
PERFORM 0970-ADD-PROG
PERFORM 0980-WRITE-REC.

```

0940-SUPERSTRUCTURE.

```

* PERFORMED BY 0930-ADD-PROJ *
IF PROJ-CODE
AND NEW-PROJ
NEXT SENTENCE
ELSE
PERFORM 0950-ADD-PROG THRU 0950-END.
GO TO 0940-END.
IF INDX-P IS EQUAL TO 100
PERFORM 0960-WRITE-REC THRU 0960-END.
GO TO 0940-END
MOVE EXP-REV-PROJ TO PROJ-TABL (INDX-P).
MOVE NO-PROG-OR-TE TO MODE-T (INDX-P).
ADD 1 TO INDX-P.
PERFORM 0950-ADD-PROG THRU 0950-END.
0940-END.
EXIT.

```

図 5.3 左側の FORTRAN のコードは十分構造化されておらず、GO TO 文によるループがある。右側のコードは、Cobalt Blue 社のリストラクチャリング・ツール FOR-STRUCT により、標準 FORTRAN-77 に準拠して構造化されたもので、GO TO 文の数も減っている。

図 5.8 Computer Data Systems 社の Retool によって、図 5.7 の COBOL コードを構造化したものの。GO TO 文は禁じられている。

図 5.9 Computer Data Systems 社の Retool でリストラクチャリングされた COBOL コード。パラグラフのイグジットへの行儀のよい分岐は許されている。

- ・ リバース・エンジニアリングツールを使用してプログラムからチャートを出力した例

```

01 PICK-LSL.
05 ORDER-NUMBER PIC XXXXX.
05 ORDER-DATE PIC DD/MM/YY.
05 PRODUCT-NUMBER PIC X(4).
05 QUANTITY-ORDERED PIC 99999.
05 QUANTITY-SHIP PIC 99999.

```

```

77 CUST-NOT-FOUND-FLAG PIC X.
77 PART-NOT-FOUND-FLAG PIC X.
77 ORDER-NOT-FOUND-FLAG PIC X.

```

```

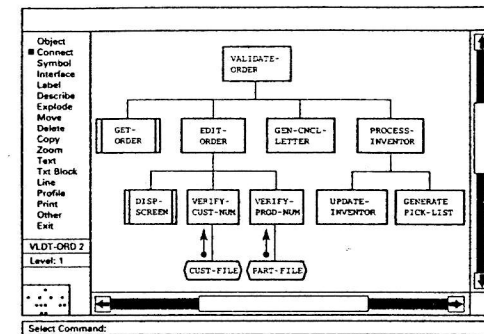
LINKAGE SECTION.
PROCEDURE DIVISION.
VALIDATE-ORDER SECTION.
* An order must be validated before it can be processed. This
* includes editing an order, generating a cancel order form, if
* required, or processing the inventory in order to ship.
CALL GET-ORDER USING ORDER-1TH.
MOVE ORDER-NUMBER TO CORDERNO.
MOVE PRODUCT-NUMBER TO CORDERNO1.
MOVE QUANTITY-ORDERED TO CORDERNO2.
PERFORM EDIT-ORDER.
PERFORM GEN-CMCL-ORDER-FORM-LETTER.
PERFORM PROCESS-INVENTORY.
STOP RUN.

```

```

EDIT-ORDER SECTION.
CALL DISP-SCREEN USING CORDERNO-SCRN.
* An order must be edited before it is accepted. This includes
* verifying the customer number and the product number to make
* sure they are valid.
PERFORM VERIFY-CUST-NUMBER.
PERFORM VERIFY-PRODUCT-NUMBER.
EXIT.

```



・ プログラムロジック
・ ファイル I/O

図 7.16 Intersolv 社の Design Recovery により、COBOL ソースコードからストラクチャチャートを作成し、Excelerator リポジトリに蓄積したもの。

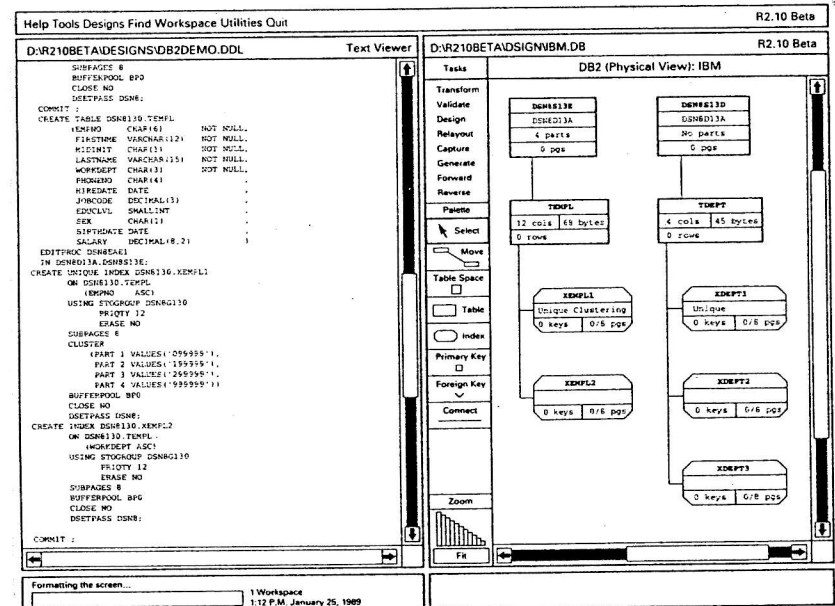


図 7.5 Bachman Information Systems 社の Re-Engineering Product Set によって、画面左側の DB2 データベースの DDL が、画面右側の物理データベースダイアグラムにリバースエンジニアリングされている。

再利用可能性

システム開発にともなう生産物を、再利用する技術である

必要性

- ・ 開発の時間と費用を削減したい。
- ・ ソフトウェアの品質を向上させたい。
- ・ 生産性を向上させたい。

再利用の形態

- ・ プロトタイプ
- ・ データ
- ・ システムおよびプログラム構造の枠組み
- ・ データモデル
- ・ プログラムコードの一部
- ・ ソフトウェアパッケージ
- ・ ソフトウェア・ライフサイクル工程

再利用をソフトウェアの方法論およびツールに組み込む

人 材

システム開発における役割をよく理解し、それをうまくこなして
く特定のプレイヤーであり、高度の教育を受けた人材である。

必要性

優れた方法論や及びツールを効果的に使用し管理する為である

必要な人材

プロジェクト・マネージャ

システム・アナリスト

システム・エンジニア

テクニカル・エンジニア

リポジトリ管理者

カットオーバー・チーム

ユーザ・チーム

システム・チャンピオン 等

管 理

すべてのことを統合し運営する仕事

必要性

例えば サッカーのチームにおいても良い選手がいてもよい監督が必要（選手一人一人をコントロールできなければならない）

- ・ 開発全体にかかわる全体の要素を理解し、計画をたて実行し評価する。また評価にもとずき再度計画、実行する。
- ・ 開発チームに対して動機づけを与える。
- ・ 評価する場合の計量の指針を持つ。
- ・ 作業環境を整える。
- ・ ユーザ役割を明確にする。
- ・ 個々の責任を明確にする。
- ・ 官僚主義を避ける。 等

